

Article

Ascon on FPGA: Post-Quantum Safe Authenticated Encryption with Replay Protection for IoT

Meera Gladis Kurian *  and Yuhua Chen * 

Department of Electrical and Computer Engineering, University of Houston, Houston, TX 77204, USA

* Correspondence: mgladiskurian@uh.edu (M.G.K.); yuhuachen@uh.edu (Y.C.)

Abstract

Ascon is a family of lightweight cryptographic algorithms designed for Authenticated Encryption with Associated Data (AEAD), hashing, and Extendable Output Functions (XOFs) in resource-constrained environments. While the AEAD variants of Ascon provide confidentiality and authenticity, they do not inherently detect replayed messages. This work presents an FPGA implementation of Ascon-128, the primary AEAD variant, on a Xilinx Artix-7 device with integrated replay detection. A 128-bit Linear Feedback Shift Register (LFSR) is used to generate a unique sequential nonce per encryption, enabling high-speed, stateless nonce generation with minimal logic complexity. At the decryption end, replay detection is performed by hashing the received nonce using Ascon-XOF128 and verifying its freshness via a Bloom Filter stored in on-chip Block RAM (BRAM). Leveraging the flexibility of Ascon-XOF128 to generate variable length outputs, our design derives all ten Bloom Filter indices from a single 256-bit XOF output using the same permutation core as the AEAD data path, thereby eliminating the need for additional hashing logic. The Bloom Filter ensures zero false negatives, and our configuration achieves a low False Positive Rate (FPR) of 0.77% theoretically and 0.17% empirically after testing 100,000 nonces, consistent with analytical models. Replay detection is fully overlapped with decryption and introduces no additional delay for messages of 64 bytes or more when using the optimized two Rounds Per Clock Cycle (RPCC) permutation core operating at 100 MHz. This architecture extends Ascon with hardware-based replay protection, offering a lightweight and scalable security solution for practical IoT deployments.



Academic Editor: Alexander Barkalov

Received: 21 May 2025

Revised: 20 June 2025

Accepted: 27 June 2025

Published: 1 July 2025

Citation: Gladis Kurian, M.; Chen, Y. Ascon on FPGA: Post-Quantum Safe Authenticated Encryption with Replay Protection for IoT. *Electronics* **2025**, *14*, 2668. <https://doi.org/10.3390/electronics14132668>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: Ascon; nonce generation; replay attack; Bloom filter; FPGA implementation; secure IoT

1. Introduction

Internet of Things (IoT) devices frequently collect, transmit, and manage data that might include personal information, financial data, health records, or other sensitive information. Many regions and industries have strict regulations and standards regarding data protection and privacy, such as the Health Insurance Portability and Accountability Act (HIPAA) in the United States and the General Data Protection Regulation (GDPR) in Europe. Implementing strong cryptographic protocols helps IoT device manufacturers and service providers comply with these regulations, avoiding legal penalties and reputational damage. Thus, the significance of secure cryptographic protocols in the IoT cannot be overstated, especially as the proliferation of IoT devices continues to grow at an unprecedented rate.

As the demand for secure communication over untrusted networks increased, especially in decentralized IoT deployments, the need for more scalable key management

led to the adoption of public-key cryptography. The advent of public-key cryptography revolutionized secure communications, allowing secure exchanges over insecure channels without a shared secret, using a pair of keys—one public, one private. One of the earliest practical implementations was RSA (Rivest–Shamir–Adleman), introduced in the late 1970s, which remains widely used for secure data transmission and digital signatures. Later, Elliptic Curve Cryptography (ECC) emerged, offering comparable security with much smaller key sizes, making it particularly suitable for resource-constrained environments such as mobile and IoT devices. Closely related to public key cryptography is the development of digital signatures, which authenticate the identity of the sender and ensure the message’s integrity, akin to a handwritten signature but far more secure. Hashing, another critical cryptographic technique, transforms input into a fixed-size string of bytes, used in data retrieval, integrity checks, and cryptographic applications. Cryptographic methods have evolved to address the needs of increasingly sophisticated digital environments, constantly adapting to new challenges, including those posed by the advent of quantum computing.

A particularly alarming issue is the “store now, decrypt later” dilemma, where attackers might capture and save encrypted data presently, planning to decrypt it later, either through conventional cryptanalysis or more critically once quantum computing has sufficiently evolved [1]. The recent advancements in quantum computing pose new challenges to traditional cryptographic methods, leading to a shift toward quantum-resistant algorithms. The rapid advancements in quantum computing, such as Microsoft’s recent unveiling of Majorana 1, the world’s first quantum chip based on the novel Topological Core architecture, highlight the acceleration toward operational quantum computers. This new processor architecture promises the potential to fit a million qubits on a single chip small enough to fit in the palm of one’s hand. This emerging quantum capability underscores the urgent need for quantum-resistant cryptographic solutions.

Ascon, a symmetric key cryptographic standard approved by the National Institute of Standards and Technology (NIST) in 2023, is engineered to provide Authenticated Encryption with Associated Data (AEAD), hashing, and Extendable Output Function (XOF) capabilities [2]. It was chosen as the primary option for lightweight authenticated encryption in the final portfolio of the Competition for Authenticated Encryption: Security, Applicability, and Robustness (CAESAR), which ran from 2014 to 2019. It is designed to be efficient in both software and hardware, making it particularly suitable for constrained devices. It offers robust side-channel resistance and resilience against misuse. However, a critical observation is that it cannot distinguish between original data messages and unauthorized retransmissions. This vulnerability to replay attacks poses a significant security risk in stateless protocols typical of IoT deployments.

This paper presents a Field Programmable Gate Array (FPGA)-based extension of the Ascon cryptographic protocol, specifically engineered to address critical security gaps in lightweight IoT environments. We focus on mitigating Ascon’s inherent vulnerability to replay attacks by implementing a hardware based nonce generation and verification framework. A 128-bit Linear Feedback Shift Register (LFSR) is deployed on a Xilinx Artix-7 FPGA to ensure per-message nonce uniqueness during encryption. At the decryption end, replay protection is enforced using a Bloom Filter-based detection system stored on FPGA Block RAM (BRAM). Filter indices are derived from the received nonce using Ascon-XOF128, which utilizes the same permutation logic as the Ascon core, effectively eliminating the overhead of implementing a separate hashing module. The Bloom Filter provides guaranteed detection of replayed messages, i.e., zero false negatives, while maintaining an exceptionally low False Positive Rate (FPR) [3], thus effectively safeguarding the integrity of IoT communications. This design is inherently scalable, efficiently adapting to diverse security requirements across low to high-end IoT devices. Furthermore, leveraging the

FPGA's intrinsic support for parallel processing and high-speed operations, our solution aligns seamlessly with IoT constraints such as limited computational resources, power efficiency, and minimal latency.

The rest of the paper is organized as follows: Section 2 reviews related work on replay attack mitigation and FPGA-based security designs. Section 3 details the methodology, including the implementation of Ascon AEAD on FPGA, with a focus on the nonce generation mechanism, followed by the integration of Bloom Filter-based replay attack detection and the design of the Bloom Filter using Ascon-XOF128 hashing. Section 4 describes the FPGA implementation. Section 5 presents the experimental results, including timing validation, permutation latency analysis, replay detection and hashing optimizations, hardware resource utilization, and a scalability analysis of the Bloom Filter-based replay detection mechanism. Section 6 provides a combined discussion and conclusion, highlighting key findings and outlining potential directions for future work.

2. Related Work

Replay protection in cryptographic systems is an active area of research, particularly in IoT deployments where constrained hardware and limited memory complicate traditional countermeasures. This section provides an overview of existing replay attack mitigation strategies used in cryptographic protocols and highlights their trade-offs in lightweight deployments. We also review recent FPGA-based security implementations, focusing on their performance optimizations and limitations in addressing replay resilience.

2.1. Replay Attack Mitigation Strategies in Cryptographic Systems

Replay attacks pose a significant threat to IoT security, where adversaries exploit the retransmission of previously intercepted messages to mislead systems into accepting outdated data. Replay attacks are relatively straightforward to execute, as they do not require any prior knowledge of the targeted system. Among the emerging cryptographic standards, Ascon stands out as a lightweight yet robust AEAD scheme designed explicitly for resource constrained environments. However, Ascon, like many cryptographic algorithms, including AES-GCM and ChaCha20-Poly1305, inherently lacks protection against replay attacks [4,5].

Several countermeasures have been proposed to mitigate replay attacks in cryptographic systems. One common strategy is timestamp-based authentication, where messages are validated based on synchronized clocks. However, as Feng et al. [6] highlight, IoT devices often lack reliable time synchronization due to clock drift, environmental variations, and energy constraints. Any desynchronization can lead to either false positives (i.e., rejecting valid messages) or false negatives (i.e., accepting replayed data), making this approach unreliable for large-scale IoT deployments.

Another widely used strategy is nonce-based authentication, where each transmitted message includes a unique, one-time-use number or nonce to ensure message freshness. Systems implementing this approach maintain a record of previously received nonces to reject duplicates [7]. While effective in theory, this method imposes substantial memory overhead, particularly in hardware-constrained IoT devices, and risks failure if nonces are reused due to poor generation or synchronization mechanisms.

A third category of replay mitigation strategies leverages challenge–response protocols and hash-chain-based mechanisms, which establish message freshness through sequential or verifiable derivations of authentication values. For instance, the Hash Media Access Control Destination Sequence Distance Vector (Hash-MAC-DSDV) scheme introduced by Adil et al. [8] utilizes MAC address registration combined with hash functions to achieve decentralized device authentication in IoT-based cyber–physical systems. While such

approaches reduce reliance on explicit nonce tracking, they often introduce non-negligible state management and cryptographic overhead, making them less suitable for stateless, lightweight encryption systems deployed on FPGAs and embedded microcontrollers with constrained resources.

2.2. Existing FPGA-Based Security Implementations and Limitations

Most FPGA-based cryptographic implementations are designed with a primary focus on optimizing encryption and decryption performance, often prioritizing throughput, low latency, and resource efficiency [9]. However, these implementations frequently overlook critical security aspects such as replay attack detection, which is essential for ensuring data integrity and preventing unauthorized retransmission of valid messages.

Several FPGA-based security architectures have been developed to support resource-constrained IoT devices, emphasizing high-speed encryption using well-established cryptographic algorithms. Notably, AES-based designs and lightweight block ciphers such as PRESENT, SPECK, and SIMON have been implemented on FPGA platforms to achieve efficient cryptographic processing [10]. These implementations leverage hardware acceleration to meet stringent real-time security requirements in IoT and embedded systems. Despite their efficiency, these designs often rely on external memory for nonce storage or depend on protocol-layer security mechanisms such as Transport Layer Security (TLS) to handle authentication and replay protection. As a result, these approaches introduce significant limitations:

- **Memory Overhead:** Many FPGA-based cryptographic systems require dedicated memory to store previously used nonces, to detect replayed messages. This approach is impractical for resource-constrained devices, as on-chip memory is limited, and off-chip storage increases power consumption and system complexity [11].
- **Protocol Dependence:** Implementations that delegate replay attack prevention to higher-layer security protocols, such as TLS or Datagram Transport Layer Security (DTLS), introduce computational and communication overhead [12]. These protocols require additional handshake mechanisms, certificate management, and session tracking, which may not be feasible in low-power real-time IoT applications.
- **Nonce Management Complexity:** Generating unique and unpredictable nonces is essential for ensuring cryptographic security. Many FPGA-based implementations rely on counters to derive nonces, but these approaches can be vulnerable if the generated values exhibit predictable patterns or periodicity. Counter-based approaches require arithmetic addition on a large number of bits, which introduces more complex logic and longer propagation delays, increasing latency and resource usage on FPGA. Additionally, improper nonce generation mechanisms may lead to reuse or synchronization issues between communicating parties, compromising security, and increasing susceptibility to replay attacks.
- **Scalability Concerns:** Traditional approaches that depend on explicit nonce-tracking mechanisms become increasingly inefficient as the number of transactions grows. Large-scale IoT networks require scalable solutions that minimize memory usage while maintaining robust security against replay attacks.

Table 1 summarizes the strengths and limitations of various replay mitigation strategies. While timestamp-based and nonce-tracking mechanisms offer basic replay resistance, they suffer from synchronization challenges or high memory demands. More advanced techniques such as Merkle tree verification and ECC-based authentication provide strong cryptographic guarantees and have seen wide adoption in cloud-based or aggregator-level smart grid systems. However, they incur substantial hardware and computational overhead [13], making them less suitable for deployment in deeply embedded or resource-constrained IoT devices such as smart meters or field sensors.

Table 1. Comparison of replay attack mitigation methods and AEAD mechanisms.

Approach/Mechanism	Principle	Advantages	Limitations
Timestamp Based [14]	Synchronized clocks	No memory overhead	IoT clock drift issues; energy constraints
Nonce-Based [15]	Tracks received nonces	Ensures uniqueness	Requires high memory storage
Hash-Based [6]	Challenge–response mechanism	No explicit nonce storage	Stateful tracking is needed
True Random Number Generator [16]	Physical entropy source	Secure nonces	Hardware overhead; unpredictable sources
Moving Target [17]	Dynamic system parameters	Harder for attackers to exploit	Computationally expensive
Merkle Tree Verification [18]	Uses hash tree of prior messages for integrity and freshness checks	Strong tamper and replay resistance; verifiable history	High storage and hashing overhead; not ideal for real-time or resource-limited systems
ECC-Tagging [19]	Error-correcting codes used to tag and verify message freshness	Robust detection of modification and replays; lightweight variants exist	Extra tag storage and ECC logic; may require per-message keying
Advanced Encryption Standard (AES)	AEAD using Galois/Counter Mode	Authenticated encryption	No built-in replay protection; depends on external nonce/session handling
Ascon	Lightweight AEAD; external nonce management	Lightweight and efficient	No inherent replay protection; requires external nonce tracking

To address this gap, the proposed system builds upon Ascon, a lightweight and NIST standardized cryptographic algorithm, offering authenticated encryption with excellent performance on FPGA platforms. Replay detection is achieved by integrating cryptographic hashing using Ascon-XOF128 with a Bloom Filter based freshness check, allowing for low latency, probabilistic replay protection with tunable FPR. Nonces are generated by a lightweight LFSR-based mechanism. Together, these components enable scalable and robust replay protection while preserving the performance and area efficiency required for low-power IoT deployments.

3. Methodology

This section details the implementation of Ascon-AEAD128 on FPGA [2], focusing on its architectural design, optimization strategies, and hardware resource management to ensure efficient encryption and authentication. An important addition to this implementation is the LFSR-based nonce generation scheme, which guarantees nonce uniqueness while maintaining a lightweight and efficient hardware footprint. Following this, we introduce our novel replay attack detection strategy using Bloom Filter, offering a probabilistic, memory-efficient alternative over conventional nonce-tracking mechanisms. The integration of Ascon-XOF128 hashing with Bloom Filter transforms nonces into secure hash mappings, eliminating the need for direct nonce storage. We discuss the system-level integration of Ascon encryption and decryption along with LFSR-based nonce generation, and Bloom Filter-based replay detection, emphasizing parallel processing, BRAM utiliza-

tion, and FPGA resource efficiency to achieve real-time attack detection in constrained IoT environments.

3.1. Ascon AEAD Implementation on FPGA

Ascon operates on a 320-bit state, which is updated using two types of permutations, denoted as p^a (a rounds) and p^b (b rounds). This 320-bit state S is divided into two components: an outer part S_r of r bits (rate) and an inner part S_c of c bits (capacity) where the values of r and $c = 320 - r$ vary depending on the specific Ascon variant. For this implementation, we adopt Ascon-AEAD128, where r equals 128. To facilitate the definition and application of round transformations, the 320-bit state S is further divided into five 64-bit words denoted as x_0, x_1, x_2, x_3, x_4 . This allows for efficient processing and manipulation of the cipher state throughout the encryption and decryption operations.

$$S = S_r || S_c = x_0 || x_1 || x_2 || x_3 || x_4 \quad (1)$$

3.1.1. Nonce Generation Mechanism

The nonce generation mechanism in this implementation utilizes a 128-bit LFSR to produce a unique nonce for each encryption session. The LFSR is initialized with a fixed seed to establish a complex starting state and is updated using a tapped feedback polynomial, where the new input bit is computed as the XOR of selected tap positions from the current register state. This structure ensures sufficient diffusion and enables a long pseudo-random sequence before repeating.

To maximize hardware efficiency and reduce latency, a one-to-many feedback configuration is employed, in which a single feedback bit is distributed to multiple tap positions. This configuration allows the logic to be implemented in just two levels, minimizing critical path delay compared to conventional many-to-one feedback designs. It is particularly advantageous for high-throughput or resource-constrained environments. An illustration of this structure is shown in Figure 1.

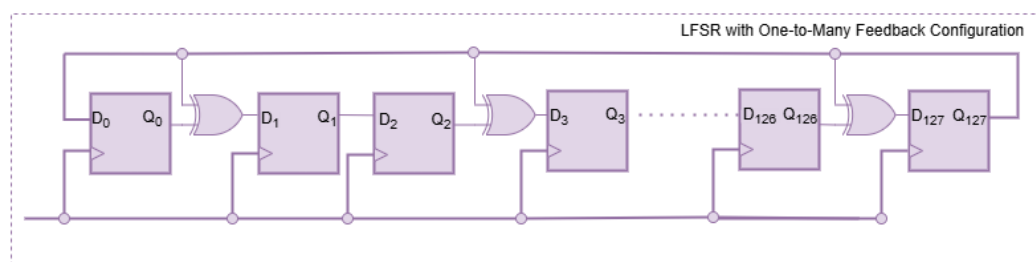


Figure 1. An illustration of one-to-many feedback configuration used to generate unique nonce values for each encryption operation.

During each clock cycle, when enabled, the LFSR shifts its state by one bit while applying the feedback logic. This produces a distinct 128-bit value for each encryption operation without requiring external randomness or counters. The design prioritizes nonce uniqueness, which is critical for ensuring authenticated encryption and preventing replay attacks. As each encryption operation uses a new LFSR state, the resulting nonce stream provides a long non-repeating sequence suitable for preventing reuse over the device's operational lifetime. In AEAD schemes like Ascon, the nonce is not a secret but must be unique for each encryption operation to ensure security. This design goal aligns with the use of an LFSR, which provides deterministic, non-repeating nonce values with minimal hardware overhead.

While LFSRs are not cryptographically secure random number generators, they are lightweight and efficient for ensuring nonce uniqueness in hardware. In this design, the

LFSR is used strictly to guarantee distinct nonces across encryption operations. Potential risks from seed reuse or LFSR periodicity are acknowledged and can be mitigated by initializing the seed from device-specific constants or startup entropy sources.

3.1.2. Authenticated Encryption and Verified Decryption

Ascon follows a sponge-based encryption approach, as shown in Figure 2, and is structured into four distinct phases—initialization, associated data processing, plaintext processing, and finalization. Its operational mode is inspired by duplex-based constructions such as MonkeyDuplex [20], but enhances security by using a stronger keyed initialization and keyed finalization function.

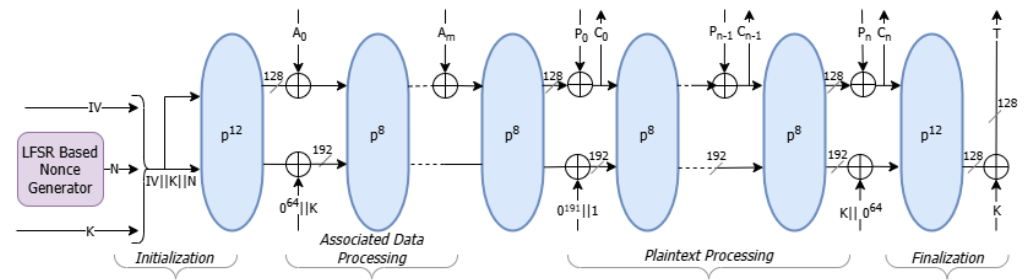


Figure 2. Authenticated encryption.

The 320-bit initial state of Ascon is formed by the secret key, K of 128 bits, and nonce N of 128 bits generated using the method elaborated in the previous section and the Initialization Vector (IV) assigned to $0x00001000808c0001$ as $S = IV || K || N$. After completing all processing stages, it produces a ciphertext C of the same length as plaintext P and a 128-bit authentication tag T :

$$\text{Ascon-AEAD128.enc}(S, A, P) = (C, T) \quad (2)$$

A twelve-round permutation, p^{12} , is first applied to the concatenated input $IV || K || N$, ensuring strong diffusion and secure mixing of the key and nonce. During the associated data absorption phase, each associated data block is XORed into the rate portion of the state, followed by an eight-round permutation, p^8 , after each block.

The plaintext processing phase absorbs the message by XORing each plaintext block into the state's rate portion, producing the ciphertext while applying an eight-round permutation function p^8 after each block. During the finalization phase, the key is reintegrated into the state, followed by another twelve-round permutation, p^{12} . The authentication tag T and ciphertext C are then extracted from the state's rate and capacity portions, respectively. This tag ensures message integrity, preventing unauthorized modifications or forgeries.

Similarly, as shown in Figure 3, the decryption function begins by initializing the 320-bit internal state as $S = IV || K || N$, where IV is a fixed constant, K is the shared secret key, and N is the nonce received alongside the ciphertext. The associated data A , ciphertext C , and authentication tag T are also provided as inputs for verification and decryption.

$$\text{Ascon-AEAD128.dec}(S, A, C, T) = \begin{cases} P, & \text{if Tag is valid} \\ \text{fail}, & \text{otherwise} \end{cases} \quad (3)$$

The decryption process mirrors the encryption phases. The associated data is first absorbed into the state using XOR operations over the rate portion, with an 8-round permutation, p^8 , applied after each block. Following that, the ciphertext blocks are processed

to recover the plaintext P , again using XOR operations, interleaved with p^8 permutations after each block.

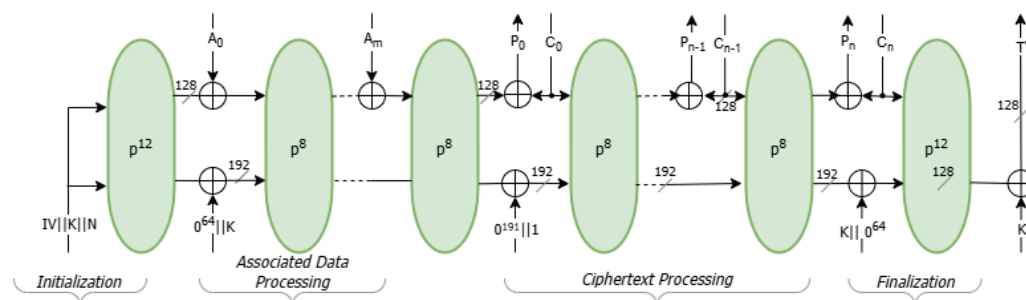


Figure 3. Authenticated decryption.

In the finalization phase, the key K is XORed back into the state, and a 12-round permutation, p^{12} , is applied. The resulting state is then used to generate a recomputed authentication tag, T' which is compared against the received tag, T . If the tags match, the decryption is deemed successful, and the original plaintext P is returned. If the tags differ, the ciphertext is considered unauthenticated, and the decryption process fails, preventing the release of invalid or potentially tampered data.

3.2. Replay Attack Detection Using Bloom Filters

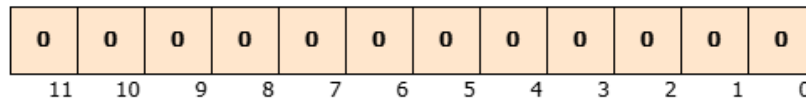
Bloom Filters are space- and time-efficient probabilistic data structures that enable fast membership checks with significantly reduced memory requirements [3]. While the standard Ascon specification does not include any built-in mechanism for replay protection, our system extends its security by integrating a Bloom Filter that probabilistically determines whether a given nonce has likely been seen before, without storing each nonce individually. This eliminates the need for full-length nonce comparisons. This approach offers substantial memory savings, requiring only a fraction of the space used by traditional error-free hashing methods, while maintaining high detection accuracy. In our design, the Bloom Filter integrates seamlessly with the Ascon-based decryption pipeline, offering scalable and efficient replay detection suitable for constrained hardware environments. Building on this foundation, we now present the underlying architecture, design choices, and operational flow of the Bloom Filter within the replay detection system, highlighting how it complements the Ascon decryption process in both functionality and efficiency.

3.2.1. Framework of Bloom Filter

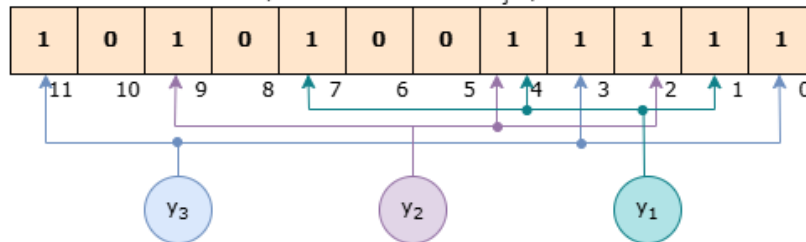
Let $S = \{x_1, x_2, \dots, x_n\}$ be a subset of a universal set U , containing n elements. A Bloom Filter represents these elements using a bit vector of length m , with all bits initially set to zero. To include an element x in S , k distinct hash functions, $\{h_1, h_2, \dots, h_k\}$, are used to assign x to k specific positions $\{h_1(x), h_2(x), \dots, h_k(x)\}$ within the bit vector, where each $h_i(x)$ falls within the range $[0, m - 1]$. The bits at these positions in the vector are then set to 1. To check if a given element is part of set S , the element is hashed to the bit vector using the same k hash functions, and the bits at the corresponding positions are examined. If any of these bits is 0, the Bloom Filter determines that the element is not part of S ; if all are 1s, the Bloom Filter suggests that the element might be in S . However, it guarantees no false negatives, meaning any element reported as “not present” is definitely not in the set. Figure 4 shows an example of a Bloom Filter with filter size, $m = 12$ bits, and hash functions, $k = 3$, used to represent a set $S = \{y_1, y_2, y_3\}$. The 12-bit vector is initialized to all zeros. Upon inserting elements, specific bits corresponding to each element are set to 1, as determined by the hash functions.

- For y_1 , suppose the hash functions determine the positions 1, 4, and 7. These bits are set to 1 in the vector.
- For y_2 , the hash functions map it to positions 2, 4, and 9. Note the shared position 4 with y_1 , showcasing hash collision.
- For y_3 , let us say the bits at position 0, 3, and 11 are set to 1.

Initial state of the BF



Inserting each element x_i into BF such that $BF[h_j(x_i)] = 1$



Query: If all $BF[h_i(x_i)] = 1$, return Positive; Else return Negative

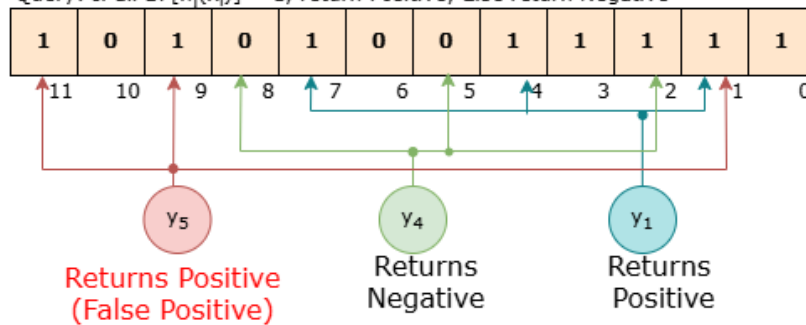


Figure 4. An illustrative example of Bloom Filter.

Now, when querying the set,

- A query for y_1 checks bits at positions 1, 4, and 7. Since all these bits are 1, the Bloom Filter returns “Positive”, correctly indicating y_1 ’s membership.
- A query for an element y_4 , which is not part of set S , might check bits at positions 2, 5, and 8. Since the bit at position 5 is 0 (assuming no previous element has affected this bit), the Bloom Filter returns “Negative”, correctly indicating that y_4 is not in the set.

However, the possibility of false positives arises:

- Suppose a query for y_5 (not in S) maps to positions 1, 9, and 11. All these positions have bits set to 1 due to the insertion of y_1 , y_2 , and y_3 . The Bloom Filter would incorrectly return “Positive”, suggesting y_5 is a member of S despite it not being true.

This example illustrates the inherent risk of false positives in Bloom Filters due to hash conflicts, where different input elements result in the same hash values affecting the same bits in the vector. Accordingly, it is essential to adopt a well-balanced configuration of Bloom Filter parameters, specifically the bit array size and the number of hash functions, to optimize the tradeoffs among memory efficiency, computational speed, and detection accuracy. While the Bloom Filter is not inherently cryptographic and has been criticized for vulnerabilities in uncontrolled environments due to its susceptibility to false positives and pollution attacks [21], it remains highly effective when deployed as an auxiliary mechanism in controlled systems [22]. In our implementation, we overcome these limitations by incorporating Ascon-XOF128, a lightweight, post-quantum secure hash function, ensuring

that the indices generated for Bloom Filter updates are tamper-resistant and difficult to predict. This combination provides a robust and efficient replay attack detection scheme suited for resource-constrained IoT environments.

3.2.2. Implementation of Bloom Filter-Based Replay Protection for Ascon on FPGA

Despite Ascon's adoption as a lightweight AEAD standard, existing FPGA implementations focus primarily on encryption efficiency, energy optimization, and side-channel resistance, with no prior work explicitly addressing replay attack mitigation in Ascon-based cryptographic systems. Traditional approaches rely on explicit nonce tracking, protocol-layer defenses, or storage-heavy mechanisms, all of which impose significant memory and computational overhead, making them unsuitable for resource-constrained FPGA-based IoT applications. To bridge this gap, we propose a novel, hardware-efficient replay detection mechanism that integrates Bloom Filters with Ascon-XOF128 hashing, providing a lightweight, scalable, and high-speed security enhancement for Ascon on FPGA.

Unlike previous methods that rely on persistent storage, our design employs a Bloom Filter to efficiently track nonces, thereby eliminating explicit memory requirements. It reuses Ascon's existing permutation modules for hashing, reducing hardware complexity and improving resource efficiency. By exploiting FPGA-level parallelism, the system achieves real-time, high-speed replay attack detection, in contrast to the latency overhead of software-based approaches. To the best of our knowledge, this work presents the first FPGA-based replay attack mitigation mechanism for Ascon, offering a novel, efficient, and scalable solution for securing IoT environments against replay attacks.

3.3. Bloom Filter Design and Setup

A Bloom Filter is a space-efficient probabilistic data structure that supports set membership queries while allowing false positives but no false negatives. The filter consists of an array of m bits, initially set to zero, and utilizes k independent hash functions to map each inserted element to k positions in the bit array. The theoretical foundation of Bloom Filters, as detailed by Tarkoma et al. [23], enables a balance between memory efficiency and query performance.

The number of bits m required for a given number of elements n and a False Positive Rate p is given by

$$m = -\frac{n \ln p}{(\ln 2)^2} \quad (4)$$

The probability of a false positive occurring after inserting n elements into the Bloom Filter can be approximated as

$$p = \left(1 - e^{-kn/m}\right)^k \quad (5)$$

The likelihood of false positives in a Bloom Filter can be minimized by choosing appropriate values for the array size m and the number of hash functions k . Increasing the number of hash functions reduces the FPR up to an optimal value k_{opt} calculated using Equation (6), beyond which additional hash functions may degrade performance by setting too many bits in the filter.

$$k_{opt} = \frac{m}{n} \ln 2 \approx \frac{9m}{13n} \quad (6)$$

A higher k also increases computational overhead, making it a trade-off between accuracy and efficiency. Similarly, expanding the filter size (m) lowers the FPR by providing more space for hash results, reducing unintended bit collisions. However, this comes at the cost of increased memory consumption, which may not be feasible in resource-constrained environments. Additionally, as the number of inserted elements grows, the probability of hash collisions rises, leading to a higher FPR [23].

In our implementation, the Bloom Filter uses 1 Megabit of on-chip BRAM, configured as a bit array of size $m = 1,048,576$ and $k = 10$ hash functions. This setup supports the tracking of approximately 100,000 nonces while maintaining an FPR below 1%. While the selected number of hash functions $k = 10$ slightly exceeds the theoretical optimum for minimizing false positives in a Bloom Filter, this choice is both deliberate and justified within the context of our hardware architecture. In conventional Bloom Filters, increasing k beyond the optimal point can lead to diminishing returns in accuracy and increased computational overhead due to the need for multiple independent hash computations.

However, we have addressed this concern through the use of Ascon-XOF128, a cryptographically secure, post-quantum resistant Extendable Output Function. It enables the generation of multiple pseudo-random outputs from a single absorbed input by incrementally squeezing the state, enabling the efficient generation of all ten Bloom Filter indices in a single hashing pass. This approach incurs minimal hardware overhead, as the hashing logic is lightweight and structurally aligned with the primary Ascon core. Furthermore, the cryptographic strength of Ascon-XOF128 guarantees high entropy and uniformity in its output distribution, reducing the likelihood of bit saturation and ensuring that Bloom Filter indices are well-dispersed, which is an essential property for maintaining a low FPR.

3.3.1. Integration of Ascon-XOF128 Hashing in Bloom Filter

Efficient hash function selection is crucial for optimizing Bloom Filter-based replay attack detection in IoT security applications. Integrating Ascon-XOF128 into Bloom Filter-based replay attack detection systems offers a balanced approach between security and performance, particularly in FPGA-based IoT environments. In this work, the replay attack detection system is designed as an extension of the existing Ascon core, making the integration of Ascon-XOF128 hashing both efficient and resource-conscious. Unlike implementations where hashing is treated as an independent module, here, Ascon-XOF128 is derived from the same permutation functions already present in the Ascon encryption-decryption core. This significantly reduces the hardware overhead. The diagram below illustrates the integration of Ascon-XOF128 as a lightweight hashing mechanism for Bloom Filter-based authentication within the extended Ascon core.

The 128-bit nonce, generated per message during encryption using a seeded LFSR, is transmitted to the decryption side and serves as the input to Ascon-XOF128, which consists of three main stages—initialization, absorbing the nonce, N , and squeezing out the hashed nonce, H , as shown in Figure 5. The Initialization Vector is defined as $IV = 0x0000080000cc0003$. Given the 128-bit nonce input, $(N \leftarrow N_0, N_1)$, the algorithm produces a 256-bit hash output structured as

$$H \leftarrow H_0 || H_1 || H_2 || H_3$$

This is partitioned into ten segments that are used as indices into the Bloom Filter for replay detection. By deriving all ten indices from a single, lightweight Ascon-XOF128 module, the design avoids redundant hashing operations, minimizes latency, and maintains a high level of resistance to collision-based attacks, all within the constraints of low-resource FPGA environments. This seamlessly integrated approach demonstrates how post-quantum cryptographic primitives like Ascon-XOF128 can be repurposed to support efficient, hardware-friendly security enhancements beyond their traditional hashing roles.

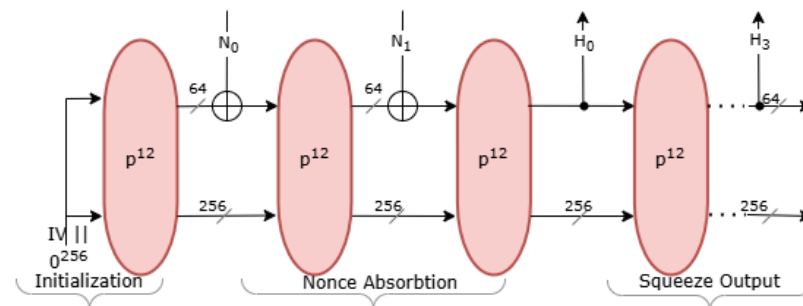


Figure 5. Structure of Ascon-XOF128 hashing used with Bloom Filter.

3.3.2. Security Strength and On-Chip Efficiency

Traditional cryptographic hash functions like SHA-256, while secure, introduce significant computational overhead, making them impractical for real-time Bloom Filter operations [24]. In contrast, non-cryptographic hash functions like MurmurHash offer superior performance but lack the necessary security guarantees, making them vulnerable in authentication-based systems.

Ascon-XOF128 offers an efficient and lightweight hashing mechanism by using the existing permutation units of the Ascon core, eliminating the need for additional hardware [2]. This integration ensures both cryptographic strength and hardware efficiency. According to the security properties summarized in Table 2, Ascon-XOF128 achieves up to 128-bit security, defined as $\min(L/2, 128)$ bits for collision resistance and $\min(L, 128)$ bits for preimage and second preimage resistance, where L is the output length. These properties are especially critical for the proposed replay detection system, which relies on Bloom Filter indexing based on hashed nonces.

Table 2. Security strengths of Ascon-based hashing algorithms (adapted from NIST documentation on Ascon hashing [25]).

Function	Output Size (bits)	Collision (bits)	Preimage (bits)	Second Preimage (bits)
Ascon-Hash256	256	128	128	128
Ascon-XOF128	L	$\min(L/2, 128)$	$\min(L, 128)$	$\min(L, 128)$
Ascon-CXOF128	L	$\min(L/2, 128)$	$\min(L, 128)$	$\min(L, 128)$

In the implemented design, each 256-bit output of Ascon-XOF128 is divided into multiple segments that determine the Bloom Filter indices. A decryption session is flagged as a replay, and plaintext release is suppressed, even when tag verification passes, if all indices derived from the session's nonce are already set in the Bloom Filter. This rule ensures accurate detection of repeated nonces, prioritizing security over false positive suppression. While this approach may lead to occasional false positives, where a fresh nonce is incorrectly flagged as reused due to the probabilistic nature of Bloom Filters, such occurrences are statistically bounded and tunable based on filter size and the number of hash segments.

An attacker aiming to exploit this mechanism would need to craft a nonce that, when processed by the XOF logic, maps to the same Bloom Filter indices as a previously stored one. However, due to the 128-bit preimage and collision resistance of Ascon-XOF128, the probability of generating such a spoofed nonce is computationally negligible. More importantly, even if such a nonce were crafted, the corresponding authentication tag would also need to match for the message to be accepted. Since tag generation depends on both the nonce and the secret key, any mismatch results in authentication failure. Thus, our system

not only prevents straightforward replay attacks but also defeats advanced forgery-based Denial-of-Service (DoS) attempts, demonstrating strong resilience in adversarial conditions.

Furthermore, Ascon-XOF128's extendable output, which allows generation of variable length hash outputs, makes it particularly well-suited for Bloom Filter indexing. The number and size of hash-derived indices can be flexibly adapted to meet target FPRs and hardware resource constraints, without the need for multiple independent hash functions. This simplifies hardware implementation, reduces logic duplication, and minimizes synchronization overhead. Combined with the use of on-chip BRAM for storing the Bloom Filter bit array, the proposed system enables high-speed lookup operations with minimal latency, avoiding external DRAM access and ensuring scalability for real-time, resource-constrained IoT deployments.

3.4. System Integration and Optimization

A core design objective is to integrate replay detection into the Ascon framework without introducing significant hardware or timing overhead. Rather than introducing an independent hashing function for the Bloom Filter indexing, this design utilizes the existing AEAD permutation logic to implement Ascon-XOF128 hash, thereby minimizing design overhead. The system workflow consists of three primary stages: Ascon encryption, replay attack detection using Bloom Filter driven by Ascon-XOF128 hashing, and Ascon decryption, as illustrated in Figure 6.

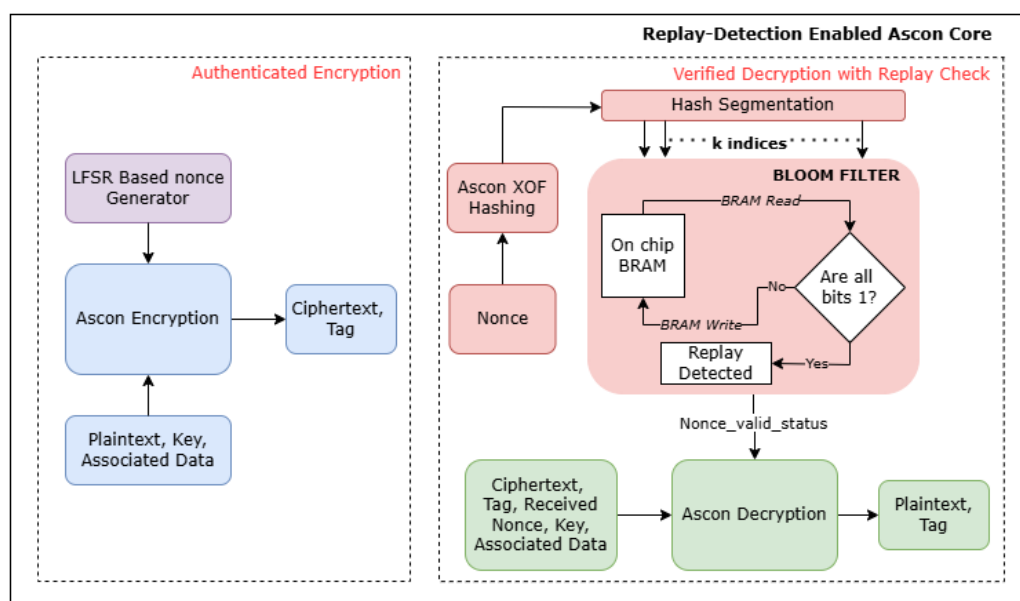


Figure 6. Block diagram of the enhanced Ascon core integrating LFSR-based nonce generation and replay detection.

In the encryption path, a 128-bit nonce is generated using an LFSR-based mechanism. This ensures a unique nonce for each encryption cycle and maintains synchronization with the Ascon encryption core. The generated nonce, along with the plaintext, key, and associated data, is fed into the Ascon authenticated encryption module, which outputs the ciphertext and a 128-bit authentication tag.

For decryption with replay detection, the received nonce is hashed using Ascon-XOF128 to produce a variable-length output. This hash is segmented into multiple parts, each of which was used to compute an index for Bloom Filter lookup. If all the derived indices are already set in the Bloom Filter, the nonce is flagged as a potential replay. Otherwise, a logic 1 is written to each of the computed indices in the Bloom Filter to mark the nonce as seen. The ciphertext is decrypted in parallel using the same nonce, key, and

associated data to recompute the authentication tag. The decrypted plaintext is released only if the authentication tag is valid and the nonce passes the replay detection check.

Thus, the proposed architecture builds on the inherent security guarantees of the Ascon-128 AEAD scheme, which ensures message integrity and authenticity through tag verification. Any tampering with the ciphertext, nonce, or associated data results in a tag mismatch, causing authentication to fail and preventing plaintext release. This behavior is preserved in the current design, which enforces tag verification alongside a nonce freshness check using a Bloom Filter, establishing a robust dual-layer security mechanism against message forgery and replay attacks.

To maintain hardware efficiency, the design integrates replay detection into the existing Ascon-based system without introducing dedicated hashing modules or redundant logic. The Bloom Filter implementation achieves a FPR below 1%, enabling reliable nonce tracking with minimal memory overhead. To mitigate the risk of DoS attacks resulting from early message rejection, decryption is initiated in parallel with nonce verification. Importantly, only authenticated messages are allowed to update the Bloom Filter, inherently preventing nonce flooding attacks from poisoning its state. Decrypted plaintext is released only after both authentication and freshness conditions are confirmed. This tightly integrated architecture ensures low-latency operation and strong security guarantees while remaining resource-efficient, making it well-suited for deployment on low-range FPGAs and is scalable to more complex platforms.

4. FPGA Implementation

We implemented the proposed system on Xilinx ARTY A7-100T FPGA, as shown in Figure 7. The design features a post-quantum-ready authenticated encryption core with hardware-level replay attack detection using nonce tracking and Bloom Filter logic. Optimization for both hardware efficiency and performance are achieved by employing lightweight cryptographic primitives, bit-sliced computation techniques, and parallel processing wherever applicable.

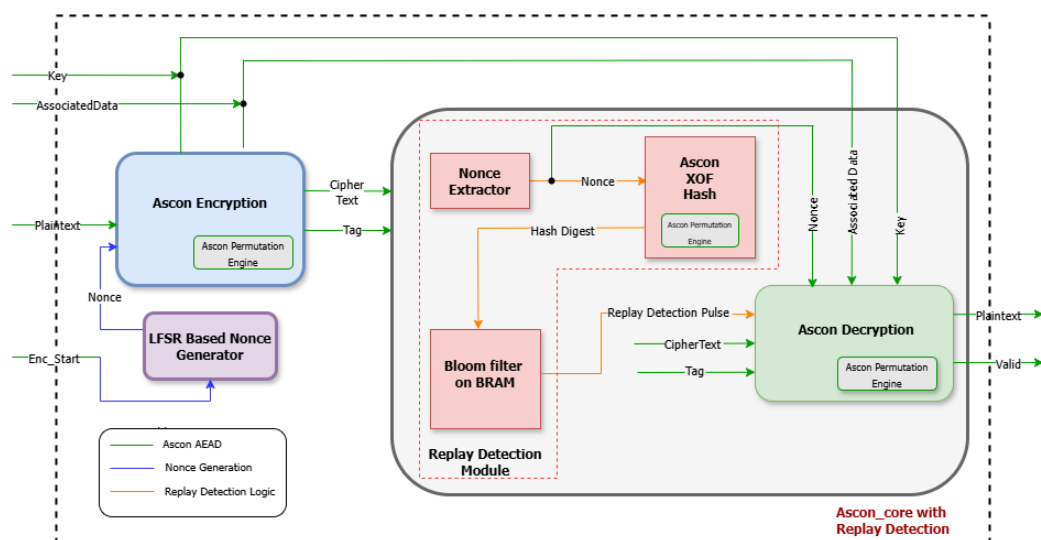


Figure 7. Ascon-based AEAD architecture with replay detection using Bloom Filter and Ascon-XOF hashing.

The Ascon core initializes a 320-bit state comprising of the secret key, the LFSR generated nonce, and an Initialization Vector that encodes algorithm parameters such as the version identifier, the number of rounds, the rate, and mode-specific constants. This state serves as the starting point for the permutation-based sponge construction. The permuta-

tions apply a round-based transformation in an iterative manner, where each round follows a Substitution–Permutation Network (SPN) structure composed of three steps—the addition of round constants p_C , a non-linear substitution layer p_S , and a linear diffusion layer p_L . Equation (1) describes the 320-bit state on which the round transformations are applied. The Finite State Machine (FSM) manages the transitions between initialization, associated data processing, plaintext/ciphertext processing, and finalization while maintaining correct control signals in the encryption steps.

The SPN structure is a self-contained algorithm that performs essential cryptographic operations such as non-linear substitution, mixing, and diffusion, making Ascon a strong candidate for secure applications in the post-quantum era. As shown in Table 3, the number of permutation rounds differs between the AEAD mode and the hashing mode, with the hashing variant requiring twelve rounds during absorption and squeezing to meet higher diffusion and uniformity requirements. In this work, the permutation core supports two hardware variations that differ in how many rounds are executed per clock cycle, enabling performance optimization based on the operation type. These variations are evaluated in the context of encryption, decryption, and nonce hashing for replay detection. Their impact on latency and hardware resource utilization is discussed in Section 5.

Table 3. Permutation round comparison for Ascon-128 AEAD and Ascon-XOF-128.

Phase	Ascon-128 (AEAD)	Ascon-XOF-128 (Hashing)
Initialization	12 rounds	12 rounds
Absorption	8 rounds	12 rounds
Squeezing	8 rounds	12 rounds
Finalization	12 rounds	

All round counts are based on the official Ascon specification.

In the round constant addition step (p_C), a predefined constant c_r as shown in Table 4 is XORed into the third 64-bit word x_2 of the 320-bit internal state S during each round. Here, i denotes the current round number (starting from 0), and r is the index used to select the appropriate round constant. For the 12-round permutation p_a , used in Ascon-128 and Ascon-XOF128 Hash, the constant is selected using $r = i$. For the reduced-round permutation p_b , applied during absorption and squeezing in Ascon-128 AEAD, the round constant index is calculated as $r = i + (a - b)$, where $a = 12$ and $b = 8$.

Table 4. Round constants c_r used in each round i of p_a and p_b .

p^{12}	p^8	Constant c_r	p^{12}	p^8	Constant c_r
0		0000000000000000f0	6	2	000000000000000096
1		0000000000000000e1	7	3	000000000000000087
2		0000000000000000d2	8	4	000000000000000078
3		0000000000000000c3	9	5	000000000000000069
4	0	0000000000000000b4	10	6	00000000000000005a
5	1	0000000000000000a5	11	7	00000000000000004b

The substitution layer, p_S , which is the S-box transformation layer, utilizes a 5-bit S-box applied in a bit sliced manner across the entire state. This S-box, detailed in Table 5, defines the core non-linear transformation.

Table 5. Ascon’s 5-bit S-box as a lookup table.

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	4	B	1F	14	1A	15	9	2	1B	5	8	12	1D	3	6	1C
x	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
$S(x)$	1E	13	7	E	0	D	11	18	10	C	1	19	16	A	F	17

The linear diffusion layer p_L ensures diffusion within each 64-bit register word x_i , significantly increasing the avalanche effect. It applies a linear function, as specified in Equation (7).

$$x_i \leftarrow x_i \oplus (x_i \ggg a_i) \oplus (x_i \ggg b_i), \quad i = 0, \dots, 4 \quad (7)$$

where the symbol $\ggg$ denotes a right rotation operation, and the rotation constants a_i and b_i for each 64-bit word x_i are defined in Table 6.

Table 6. Rotation constants used in the linear diffusion layer of Ascon permutation.

Word (x_i)	Rotation Constant a_i	Rotation Constant b_i
x_0	19	28
x_1	61	39
x_2	1	6
x_3	10	17
x_4	7	41

In the proposed system, a 128-bit nonce is generated using an LFSR before each encryption operation. Both the plaintext and the associated data are processed in 128-bit blocks, consistent with the Ascon-128 specification. After the final permutation, the ciphertext is extracted from the state, and a 128-bit authentication tag is derived to ensure message authenticity and integrity.

In the decryption phase, the received ciphertext and associated data are processed in 128-bit blocks, consistent with the encryption procedure. Simultaneously, the 128-bit nonce associated with the message is passed through the Ascon-XOF128 hashing module to generate a 256-bit digest. This digest is segmented into ten indices, each of which is used to query a Bloom Filter stored in on-chip BRAM for replay detection. While decryption begins in parallel, plaintext is withheld unless the authentication tag is successfully verified and the nonce is confirmed to be fresh. If all ten indices derived from the nonce are set in the Bloom Filter, the message is flagged as a potential replay, and the plaintext is securely discarded even if tag verification succeeds. Otherwise, the nonce is inserted into the Bloom Filter for future comparison, and the authenticated plaintext is released.

Figure 8 illustrates the FSM that governs the replay detection process using a Bloom Filter. The system begins in the IDLE state, waiting for a hash_start signal indicating that a new nonce is available for evaluation. Upon receiving this signal, it transitions to the START_HASH state, where Ascon-XOF128 hashing is triggered on the received 128-bit nonce. The FSM then enters the WAIT_HASH_DONE state, holding until XOF hashing completes. Once the digest is ready, the system proceeds to GEN_INDICES, where the 256-bit output of Ascon-XOF128 is segmented into ten index values corresponding to positions in the Bloom Filter. In the BF_CHECK state, the system sequentially reads BRAM at the ten derived indices and collects the corresponding bit values. These bits are evaluated in the REPLAY_EVAL state using a logical AND operation. If all bits are set to 1, the nonce is considered previously seen, and a replay is flagged; otherwise, the nonce is considered fresh.

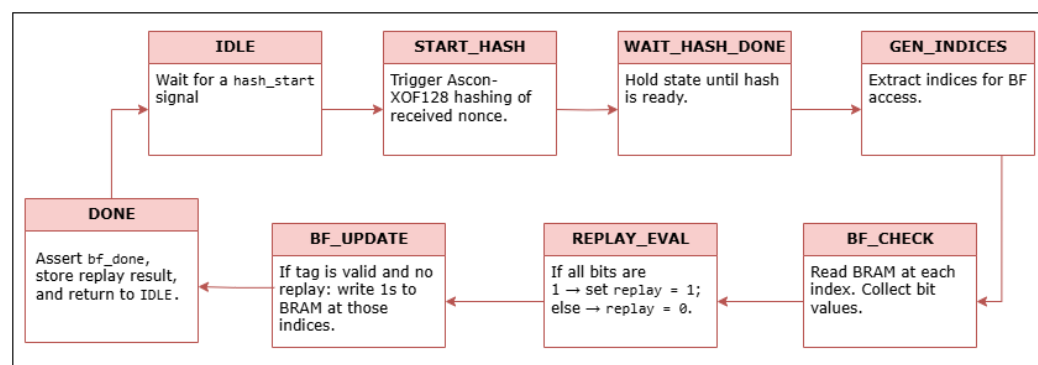


Figure 8. FSM for replay detection.

If the tag associated with the message is valid and no replay is detected, the FSM transitions to the **BF_UPDATE** state, where a 1 is written to each of the ten Bloom Filter indices to record the nonce as seen. This update occurs only for fresh, authenticated messages, ensuring that replayed or unauthenticated inputs do not contaminate the filter. This update process is non-blocking and occurs in the background, as the system does not initiate the next replay check until nonce hashing for the subsequent message has completed. As measured in simulation, the Bloom Filter update process completes significantly faster than the nonce hashing operation. This temporal gap ensures that all filter updates safely complete before the next replay check begins, even when processed in the background. Finally, the system enters the **DONE** state, where the `bf_done` signal is asserted and the replay result computed prior to the update phase is communicated to the top-level controller before returning to **IDLE**. This standalone FSM enables low-latency, parallel replay detection by efficiently reusing the existing Ascon permutation core and performing all checks entirely in hardware, ensuring that only messages with fresh nonces are permitted for further processing.

Although a traditional hash table is not implemented, the Bloom Filter combined with Ascon-XOF128 hashing provides equivalent functionality for tracking previously seen nonces in a lightweight and memory-efficient manner. The hashed indices serve as a compact representation for freshness checking without requiring explicit key-value storage. In practical IoT deployments, the nonce, ciphertext, and authentication tag can be transmitted over serial interfaces such as UART or SPI and parsed into 128-bit blocks by the hardware modules. This design ensures compatibility with standard IoT communication protocols while maintaining low area and latency overhead.

5. Results

This section evaluates the proposed FPGA-based Ascon architecture across several dimensions, including functional correctness, latency performance, and hardware efficiency. We begin with simulation-based validation of the replay detection mechanism, followed by a detailed analysis of permutation latency, timing synchronization, and Bloom Filter operations. The scalability and resource footprint of the system are assessed to demonstrate its suitability for low-power and high-throughput IoT deployments.

5.1. Experimental Validation

Figure 9 illustrates a simulation of the test sequence, conducted at a 100 MHz clock frequency, to validate the effectiveness of the proposed replay attack detection mechanism in lightweight IoT security applications. It includes two scenarios.

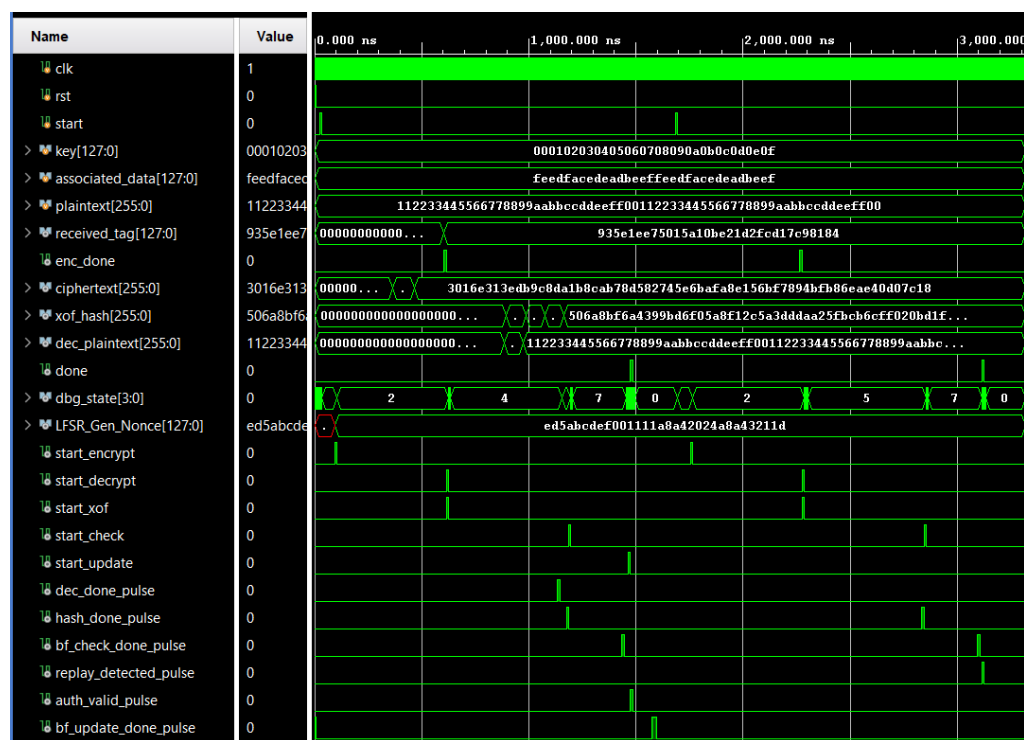


Figure 9. Simulation waveform illustrating the replay attack detection mechanism in action.

In scenario 1, a valid message is encrypted using a freshly generated nonce by an LFSR. Upon receiving the start signal, the FSM transitions from the IDLE state to WAIT_NONCE, where it waits for the LFSR to generate a valid nonce. Once available, the nonce is latched, encryption begins, and the FSM moves to WAIT_ENC. After encryption completes (enc_done), both decryption and hashing are triggered in parallel as the FSM progresses through WAIT_DEC and WAIT_XOF. After decryption (dec_done_pulse), the decrypted plaintext and computed authentication tag are buffered. Once hashing completes (hash_done_pulse), the FSM proceeds to BF_CHECK, where the Bloom Filter is checked for nonce freshness. In the DECISION state, the FSM evaluates both the tag match and the Bloom Filter output. Since the tag is valid and the nonce is fresh, authentication passes, and auth_valid_pulse is asserted. The authenticated plaintext is released immediately as the FSM transitions to the DONE state. Simultaneously, the system triggers the BF_UPDATE phase, where a 1 is written to each of the ten Bloom Filter indices to record the nonce as seen. This update is performed in the background and completes independently, without stalling the main control flow.

In scenario 2, the same LFSR-generated nonce from scenario 1 is intentionally reused to emulate a replay attack. After encryption, decryption and nonce hashing begin in parallel, as before. Although the authentication tag again matches, the Bloom Filter detects that the nonce has already been used. During the REPLAY_EVAL phase, this replay is flagged and latched internally (replay_detected_pulse). In the subsequent DECISION state, the FSM evaluates the tag match and the replay detection result. Since the nonce is no longer fresh, the FSM bypasses the Bloom Filter update phase and transitions directly to DONE without asserting auth_valid_pulse. As a result, the plaintext output is suppressed. This sequence confirms that both a valid authentication tag and a fresh nonce are required for successful message acceptance. Two independent FSMs manage decryption and nonce hashing in parallel, with the top-level controller ensuring that Bloom Filter-based replay detection is triggered immediately after hashing completes. This design minimizes authentication latency without compromising security.

Figure 10 shows the hardware behavior of the Ascon-based IoT authentication system during a replay detection test. This configuration is intended solely for verification and demonstration and does not reflect the operational setup, where detection signals are integrated with high-speed data handling modules. In the first execution (left), a unique nonce is used, the authentication passes, and the `auth_valid` LED (green) lights up, confirming the message integrity and `dec_done` LED (blue) to indicate valid decryption. In the second execution (right), the same nonce is reused, triggering replay attack. The replay LED (red) lights up while the `auth_valid` and `dec_done` LEDs remain off. This hardware demonstration validates the system's ability to detect replay attempts and ensure message freshness in real time, an essential feature for secure IoT authentication. These observed LED outcomes correspond directly to the internal FSM states and replay detection signals shown in Figure 9. Specifically, the `auth_valid_pulse`, `dec_done_pulse`, and `replay_detected_pulse` signals are routed to drive the green, blue, and red LEDs, respectively.

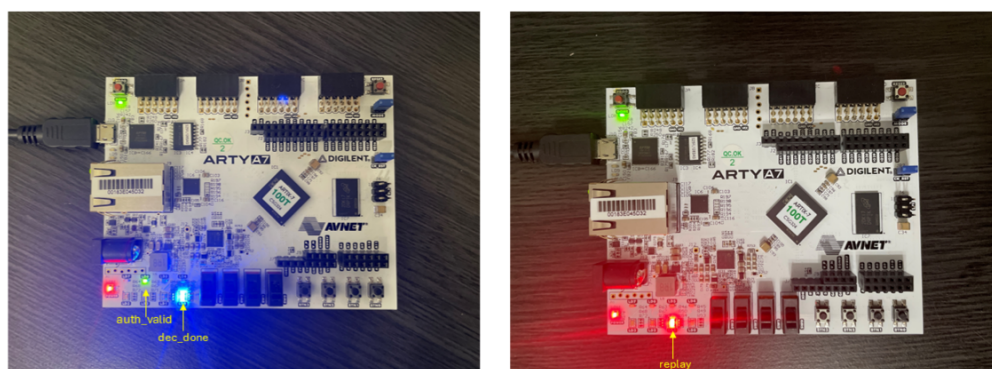


Figure 10. FPGA board output demonstrating replay detection test via LED indicators.

5.2. Permutation Latency Analysis

To evaluate the efficiency of the Ascon permutation under different configuration strategies, two designs were implemented and compared based on how permutation rounds are distributed across clock cycles. The one Round Per Clock Cycle (RPCC) design achieves a balance between latency and resource usage by executing each round in a single cycle that integrates round constant addition, substitution, and linear diffusion. In contrast, the 2RPCC design offers the lowest latency by cascading two full rounds within a single cycle. This optimized version reduces the total cycle count while introducing a moderate increase in combinational logic complexity, a tradeoff that enables faster hashing performance [26].

Table 7 summarizes the latency for both 12-round and 8-round permutations under each configuration. The 2RPCC configuration achieves the lowest overall latency and is preferred for the Ascon-XOF128 hashing variant for the Bloom Filter, while the 1RPCC version provides a favorable trade-off between speed and resource utilization for authenticated encryption and decryption.

This distinction arises from different data handling characteristics of these operations: the Ascon-XOF128 module processes a fixed 128-bit nonce, which allows two permutation rounds to be unrolled and executed in a single clock cycle without timing closure issues. This makes the 2RPCC configuration highly effective for nonce hashing in replay detection. Conversely, encryption and decryption handle variable-length messages, such as 200-byte payloads, and must process many sequential 128-bit blocks. In such scenarios, the 1RPCC schedule is more suitable, as it supports higher clock frequencies, reduced resources, better timing closure, and consistent throughput for streaming multi-block messages. This makes

it an optimal choice for scalable, high-throughput authenticated encryption on resource-constrained IoT-class FPGAs.

Table 7. Ascon permutation latency under different configuration strategies.

Permutation Type	12 Rounds (ns)	8 Rounds (ns)	Comments
1RPCC	130	90	Balanced latency and resource usage; suitable for AEAD core
2RPCC	70	50	Fastest implementation; latency-optimized for Ascon-XOF hashing

5.3. Replay Detection Timing and Hashing Optimization

Replay detection performance was evaluated by analyzing the latency of nonce hashing and Bloom Filter verification relative to the decryption path. In the proposed architecture, nonce hashing is accelerated using a 2RPCC permutation core. Rather than executing one permutation round per cycle as in a standard Ascon-XOF128 configuration, the 2RPCC module performs two rounds per cycle using fully combinational logic. This enables the permutations required for nonce hashing to complete in approximately 560 ns. The subsequent Bloom Filter check adds 250 ns. Due to FSM transitions and control logic overhead, the total measured replay detection latency is approximately 820 ns, slightly exceeding the analytical sum.

In the implemented design, Bloom Filter updates are non-blocking and proceed in parallel with the start of the next nonce hashing cycle. Once a replay check passes, the Bloom Filter begins updating while the system prepares for the next input. This overlap ensures that the update process does not introduce additional latency or stall the replay detection pipeline, allowing continuous operation without interrupting throughput. A detailed latency breakdown for all stages, including a comparison with the 1RPCC hashing configuration at 100 MHz clock, is provided in Table 8. Since the latency in clock cycles remains fixed, the absolute time increases proportionally at lower frequencies and decreases at higher ones. This frequency dependence allows system designers to tune clock rates according to power and performance needs, while maintaining the correctness and synchronization of the replay detection process.

Table 8. Latency summary for Ascon replay-resistant architecture (32-byte plaintext, 16-byte associated data).

Operation or Stage	Latency (ns)	Clock Cycles	Input Size
Encryption	510	51	32B PT + 16B AD
Decryption	520	52	32B CT + 16B AD
Nonce Hashing *	560	56	16B Nonce (2RPCC)
	920	92	16B Nonce (1RPCC)
Bloom Filter Check *	250	25	10 indices
Bloom Filter Update	110	11	10 indices

* Nonce hashing and Bloom Filter check are executed in parallel with decryption. All values assume a 100 MHz clock.

Since replay detection latency is independent of message size, the decryption path can avoid stalling as long as the input data meets the required minimum threshold. Based on a simulated decryption latency of approximately 520 ns for a 32-byte plaintext with 16-byte associated data, measured at 100 MHz, this implies a minimum plaintext size of 64 bytes (four 128-bit blocks) is required to match the 820 ns replay detection latency when using the 2RPCC hashing configuration. For the same plaintext and associated data size, if hashing

were implemented with a 1RPCC core instead, the replay detection latency would increase to approximately 1180 ns, requiring a minimum plaintext size of 80 bytes (five blocks) to avoid stall in the data path. These values are summarized in Table 9, providing guidance for latency-aware system design in secure IoT applications.

Table 9. Replay detection latency and minimum plaintext size to avoid decryption stalling *.

Hashing Configuration	Replay Detection Latency (ns)	Minimum Plaintext Size (bytes)
1RPCC	1180	80
2RPCC	820	64

* All values assume a 100 MHz clock.

This latency threshold aligns well with typical IoT message sizes in applications such as smart meters, wearables, and industrial sensor nodes, where payloads commonly range from 100 to 200 bytes. These values are consistent with protocol constraints found in lightweight IoT communication standards. For example, LoRaWAN supports maximum uplink payloads ranging from 51 to 222 bytes depending on region and data rate [27], while MQTT and MQTT-SN protocols allow for small, frequent sensor messages, with MQTT supporting payloads up to 256 MB depending on negotiated packet limits [28,29]. As such, the proposed design remains broadly compatible with a wide spectrum of real world IoT deployments. These results confirm that the implemented design supports parallel, high-throughput, and replay-resilient authenticated message processing on FPGA hardware.

5.4. Hardware Resource Utilization

The hardware resource utilization of the complete Ascon-based authentication and replay detection system under three different architectural configurations is summarized in Table 10. The design was synthesized for a Xilinx Arty A7-100T FPGA, operating at a clock frequency of 100 MHz. All configurations include the same top-level functionality, integrating modules for authenticated encryption (AEAD), nonce generation, nonce hashing, and replay detection using a Bloom Filter.

Type 1 employs a 1RPCC permutation across the encryption, decryption, and hashing cores; Type 2 uses 1RPCC for encryption and decryption but adopts a 2RPCC configuration for the hashing core to reduce verification latency; and Type 3 applies a 2RPCC permutation for all three cryptographic cores.

Across these implementations, the top-level resource utilization ranges from 3490 to 5335 Slice LUTs (5.5–8.4%) and 4285 to 4287 Slice Registers (3.38%). The Bloom Filter controller consistently uses 32 BRAM tiles, representing 23.7% of the available BRAM. No UltraRAM or DSP resources are consumed. The architecture demonstrates a flexible and compact hardware footprint, allowing trade-offs between area and performance depending on deployment constraints.

The total on-chip power was measured as 208 mW, 258 mW, and 377 mW for the Type 1, Type 2, and Type 3 configurations, respectively. The corresponding dynamic power values were 109 mW, 159 mW, and 277 mW respectively. These results reflect the increased combinational logic activity in the higher-throughput 2RPCC configuration and illustrate the trade-off between latency and energy efficiency. This architectural flexibility enables deployment on a wide range of low to mid-tier FPGAs, with configurations tailored to the energy and performance requirements of specific IoT applications.

Using a 1RPCC permutation for both AEAD and hashing in the replay detection system provides a highly area-efficient solution. This configuration is well-suited for low-power or resource-constrained systems where performance requirements are moderate. At the same time, as seen in the Type 2 configuration, the 2RPCC hashing design offers significant latency benefits while maintaining a modest resource profile, making it a practical option

when higher throughput or faster tag verification is required. This architectural flexibility allows system integrators to trade off area and latency based on specific application needs.

Table 10. FPGA resource utilization across three Ascon configurations: Type 1 (1RPCC AEAD + 1RPCC Hash), Type 2 (1RPCC AEAD + 2RPCC Hash), and Type 3 (2RPCC AEAD + 2RPCC Hash).

Module	LUTs Type 1	LUTs Type 2	LUTs Type 3	Slice Regs Type 1	Slice Regs Type 2	Slice Regs Type 3	BRAM Tiles
Top-level Module	3490	4097	5335	4285	4285	4287	32
Nonce Generator	138	138	138	390	390	390	0
Encryption Core	1125	1125	1744	1375	1375	1376	0
Decryption Core	1204	1204	1821	1116	1116	1117	0
Nonce Hasher (Ascon-XOF128)	847	1453	1453	1185	1185	1185	0
Bloom Filter Controller	168	171	171	71	71	71	32

Note: Permutation optimizations (1RPCC/2RPCC) apply only to encryption, decryption, and hashing cores. The nonce generator and Bloom Filter controller are not affected by the RPCC configuration.

It is also important to note that the total resource utilization of the top level module is not a strict sum of the individual submodules. Additional control logic, such as Finite State Machines, signal routing, and orchestration logic, contributes to the overall footprint. Moreover, FPGA synthesis optimizations, including resource sharing and boundary logic absorption, may cause the final resource count to differ slightly from the sum of its parts. Overall, the design achieves an efficient and scalable partitioning of functionality, maintaining a compact resource profile while supporting secure, high-throughput authenticated communication.

Table 11 compares our architecture with representative Ascon hardware implementations. While prior works primarily focus on performance and area, they do not address replay protection. Moreover, due to differences in FPGA platforms and inconsistent reporting of power consumption and message size, a fully equivalent comparison is not feasible. Our implementation uniquely integrates real-time replay detection using a Bloom Filter, while maintaining a compact hardware footprint on an Artix-7 platform.

Table 11. Comparison with prior Ascon FPGA implementations.

Implementation	Device	LUTs	Freq. (MHz)	Replay Protection
This Work (Type 1)	Artix A7-100T	3490	100	✓
Ascon-128 [30]	Spartan-6	1985	96.89	✗
Ascon-128 [31]	Spartan-6	2781	129.94	✗
Ascon-128 [30]	Virtex-7	1723	173.7	✗
Ascon-128 [32]	Kintex-7	2809	181.82	✗

5.5. Scalability Analysis of Bloom Filter Replay Detection

To evaluate the scalability of the proposed replay detection system, we analyze the Bloom Filter's theoretical capacity using the standard FPR model. The Bloom Filter is configured with a size of 1 Mbit and utilizes $k = 10$ hash functions derived from the 256-bit Ascon-XOF128 output. This estimation, based on Equation (5), assumes independent and uniformly distributed hash functions and no bit-level errors. These are conditions commonly accepted in Bloom Filter applications. The use of Ascon-XOF128, an NIST-standard lightweight Extendable Output Function, provides high entropy and strong diffusion, aligning well with the theoretical assumptions. This ensures that the Bloom

Filter indices are evenly distributed, which is critical to minimizing FPR and maintaining predictable performance.

To empirically validate these assumptions, we conducted using Python 3.11.13 simulation comparing two nonce generation strategies: a simple counter and the proposed hardware-oriented 128-bit LFSR. In both cases, nonces were hashed using Ascon-XOF128 to generate Bloom Filter indices. Table 12 summarizes theoretical design parameters, while Table 13 presents the experimental results from Python simulation. These collectively confirm the system’s scalability and robustness in replay protection, especially under high-throughput conditions.

For up to 100,000 unique nonce insertions, both methods maintained an FPR well below 1%, consistent with theoretical predictions. To evaluate behavior beyond the nominal capacity, the test was extended to 200,000 total queries. After 193,191 insertions, the counter-based approach yielded an FPR of 3.40%, while the LFSR-based method resulted in 3.37%. These findings confirm that Ascon-XOF128’s strong diffusion produces uniformly distributed indices regardless of nonce structure, validating the suitability of LFSR as a lightweight hardware nonce generator.

Table 12. Theoretical Bloom Filter capacity for FPR < 1% under different configurations.

Bloom Filter Size (bits)	Hash Count (k)	Max Nonces (n)	FPR
1,048,576 (1 Mbit)	8	100,000	0.0066
1,048,576 (1 Mbit)	10	100,000	0.0077
1,048,576 (1 Mbit)	12	98,000	0.0088
2,097,152 (2 Mbit)	10	200,000	0.0077

Table 13. Empirical FPR comparison: LFSR vs counter-based nonces with Ascon-XOF.

Total Tests	LFSR-Based Nonces		Counter-Based Nonces	
	Insertions	FPR	Insertions	FPR
100,000	99,834	0.00166	99,837	0.00163
120,000	119,503	0.00414	119,544	0.00380
140,000	138,889	0.00794	138,897	0.00788
160,000	157,749	0.01407	157,725	0.01422
180,000	175,934	0.02259	175,856	0.02302
200,000	193,269	0.03366	193,191	0.03405

The current Bloom Filter configuration supports up to 100,000 unique nonces with an FPR of 0.17%. However, continued insertions beyond this capacity, as illustrated by our empirical test, lead to filter saturation and a gradual increase in FPR. This is a well-documented limitation of fixed-size Bloom Filters in long-running deployments. To address this in practice, strategies such as periodic filter resets, rotation, or time-based partitioning can be employed depending on application and security requirements [23]. However, periodic resets inherently discard prior entries, which may permit undetected replay of older messages. Future work will further evaluate long-term FPR behavior under varying nonce distributions and saturation scenarios.

While the current implementation uses a fixed size Bloom Filter, future iterations may incorporate adaptive structures such as counting filters, time-based aging, or sliding windows to maintain replay detection performance under prolonged high-throughput

operation. These mechanisms can help mitigate saturation without requiring full resets, especially in resource-constrained devices with continuous communication.

The actual nonce requirement depends heavily on device behavior. High-throughput devices (e.g., smart meters, industrial sensors) may quickly saturate the Bloom Filter, while low-duty-cycle nodes (e.g., wearables, environmental sensors) may operate within capacity over long durations. For these cases, session-based reset policies aligned with key rotation can be employed. Since nonce tracking is only valid within a key epoch, resetting the Bloom Filter alongside key updates avoids stale data accumulation and ensures effective replay detection. These system-level strategies maintain security while keeping memory use minimal, demonstrating the proposed method's practicality for diverse IoT deployments.

Importantly, the proposed design ensures that only authenticated messages, i.e., those passing Ascon tag verification, result in the nonce being inserted into the Bloom Filter. As such, even if an adversary attempts to flood the system with spoofed messages containing randomly generated nonces, these nonces will not be recorded unless the corresponding ciphertext passes authentication. Given the strength of Ascon's tag validation, the likelihood of a forged message being accepted is negligible. This design choice mitigates DoS attempts via nonce flooding, as unauthorized or unauthenticated messages cannot contribute to Bloom Filter saturation. Consequently, the filter maintains its integrity and performance, even under adversarial conditions.

Additionally, the architecture is well-suited for extension to multi-core or distributed deployments, where synchronized hardware nonce generation across nodes can ensure system-wide protection against replay attacks. These directions offer pathways for further scaling the design to match the evolving demands of next-generation IoT and edge-computing systems.

6. Conclusions

This work presents a lightweight, FPGA-based implementation of Ascon-128 with integrated replay attack detection using a Bloom Filter. The proposed architecture addresses a key limitation of the standard Ascon specification, its inability to distinguish between original messages and unauthorized replays, by introducing a hardware-based replay protection layer. The design leverages a 128-bit LFSR-based nonce generator, which produces a unique nonce per encryption operation. Unlike counter-based methods that require persistent state, the LFSR offers a lightweight, stateless hardware solution that maintains uniqueness without relying on external entropy sources. Replay detection is achieved by hashing each nonce using Ascon-XOF128 and mapping the output to a Bloom Filter stored in on-chip BRAM. The hashing utilizes the same permutation core as the main AEAD datapath, minimizing hardware duplication and ensuring a compact implementation. Session isolation is enforced by resetting the Bloom Filter when the session key changes, preventing stale entries from persisting across authentication sessions. This tightly integrated approach maintains low latency and achieves reliable, low-overhead replay detection, with an FPR below 1%, where the theoretical and empirical values are 0.77% and 0.17%, respectively, based on testing with 100,000 nonces. This enhancement is achieved without modifying the Ascon-128 AEAD algorithm, preserving NIST compliance while extending its capabilities. The replay protection module and nonce generator act as modular extensions, providing a lightweight and scalable plug-in that enhances Ascon-128 from a lightweight encryption scheme into a secure authentication framework with integrated replay detection. The architecture supports a range of configurations, enabling trade offs between latency and area efficiency. Overall, the proposed design is well-suited for low-power, regulation-compliant IoT applications that require secure and efficient message authentication at scale.

Author Contributions: Conceptualization, M.G.K. and Y.C.; methodology, M.G.K. and Y.C.; investigation, M.G.K. and Y.C.; writing—original draft preparation, M.G.K.; writing—review and editing, M.G.K. and Y.C.; supervision, Y.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding authors.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

FPGA	Field Programmable Gate Array
IoT	Internet of Things
LFSR	Linear Feedback Shift Register
BRAM	Block Random Access Memory
NIST	National Institute of Standards and Technology
FPR	False Positive Rate
XOF	Extendable Output Function
AES	Advanced Encryption Standard
SPN	Substitution–Permutation Network
RPCC	Round Per Clock Cycle

References

1. Singh, H. Managing the Quantum Cybersecurity Threat: Harvest Now, Decrypt Later. In *Quantum Computing*; CRC Press: Boca Raton, FL, USA, 2024; pp. 142–158.
2. *NIST Special Publication 800-232 (Initial Public Draft)*; Technical Report; U.S. Department of Commerce: Washington, DC, USA, 2024. [CrossRef]
3. Bloom, B.H. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* **1970**, *13*, 422–426. [CrossRef]
4. Böck, H.; Zauner, A.; Devlin, S.; Somorovsky, J.; Jovanovic, P. {Nonce-Disrespecting} adversaries: Practical forgery attacks on {GCM} in {TLS}. In Proceedings of the 10th USENIX Workshop on Offensive Technologies (WOOT 16), Austin, TX, USA, 8–9 August 2016.
5. Joux, A. Comments on the Draft GCM Specification—Authentication Failures in NIST Version of GCM. 2006. Available online: http://csrc.nist.gov/groups/ST/toolkit/BCM/documents/comments/800-38_Series-Drafts/GCM/Joux_comments.pdf (accessed on 26 August 2024).
6. Feng, Y.; Wang, W.; Weng, Y.; Zhang, H. A replay-attack resistant authentication scheme for the internet of things. In Proceedings of the 2017 IEEE International Conference on Computational Science and Engineering (CSE) and IEEE International Conference on Embedded and Ubiquitous Computing (EUC), Guangzhou, China, 21–24 July 2017; Volume 1, pp. 541–547.
7. Sella, Y.; Smith, P.; Dagan, T.; Fraenkel, I. Anti-Replay Counter Measures. WO2013128317A1, 13 February 2013. Available online: <https://patents.google.com/patent/WO2013128317A1/en> (accessed on 24 April 2025).
8. Adil, M.; Jan, M.A.; Mastorakis, S.; Song, H.; Jadoon, M.M.; Abbas, S.; Farouk, A. Hash-MAC-DSDV: Mutual Authentication for Intelligent IoT-Based Cyber–Physical Systems. *IEEE Internet Things J.* **2022**, *9*, 22173–22183. [CrossRef] [PubMed]
9. Yazdeen, A.A.; Zeebaree, S.R.; Sadeeq, M.M.; Kak, S.F.; Ahmed, O.M.; Zebari, R.R. FPGA implementations for data encryption and decryption via concurrent and parallel computation: A review. *Qubahan Acad. J.* **2021**, *1*, 8–16. [CrossRef]
10. Diehl, W.; Farahmand, F.; Yalla, P.; Kaps, J.P.; Gaj, K. Comparison of hardware and software implementations of selected lightweight block ciphers. In Proceedings of the 2017 27th International Conference on Field Programmable Logic and Applications (FPL), Ghent, Belgium, 4–8 September 2017; pp. 1–4.
11. McKay, K.; Bassham, L.; Turan, M.S.; Baish, M.; Boyle, M. *Lightweight Cryptography for the Internet of Things*; Technical Report NIST IR 8114; National Institute of Standards and Technology (NIST): Gaithersburg, MD, USA, 2017.
12. Restuccia, G.; Tschofenig, H.; Baccelli, E. Low-power IoT communication security: On the performance of DTLS and TLS 1.3. In Proceedings of the 2020 9th IFIP International Conference on Performance Evaluation and Modeling in Wireless Networks (PEMWN), Berlin, Germany, 1–3 December 2020; pp. 1–6.

13. Liu, Y.; Cheng, C.; Gu, T.; Jiang, T.; Li, X. A Lightweight Authenticated Communication Scheme for Smart Grid. *IEEE Sens. J.* **2016**, *16*, 836–842. [CrossRef]
14. Sandosh, S.; Saxena, R.; Shah, S.; Rachiraju, S.S. State-of-the-Art of Voice Assistance Technology, Mitigating Replay Attacks: A Comprehensive Discussion. In Proceedings of the 2024 5th International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), Tirunelveli, India, 11–12 March 2024; pp. 594–601.
15. Vlot, M.; Schaaf, C. Replay Attack Prevention for Content Streaming System. U.S. Patent 10,025,911, 17 July 2018.
16. Gope, P.; Lee, J.; Quek, T.Q. Lightweight and practical anonymous authentication protocol for RFID systems using physically unclonable functions. *IEEE Trans. Inf. Forensics Secur.* **2018**, *13*, 2831–2843. [CrossRef]
17. Weerakkody, S.; Sinopoli, B. Detecting integrity attacks on control systems using a moving target approach. In Proceedings of the 2015 54th IEEE Conference on Decision and Control (CDC), Osaka, Japan, 15–18 December 2015; pp. 5820–5826.
18. Li, H.; Lu, R.; Zhou, L.; Yang, B.; Shen, X. An Efficient Merkle-Tree-Based Authentication Scheme for Smart Grid. *IEEE Syst. J.* **2014**, *8*, 655–663. [CrossRef]
19. Hammi, B.; Fayad, A.; Khatoun, R.; Zeadally, S.; Begriche, Y. A Lightweight ECC-Based Authentication Scheme for Internet of Things (IoT). *IEEE Syst. J.* **2020**, *14*, 3440–3450. [CrossRef]
20. Daemen, J.; Bertoni, G.; Peeters, M.; Van Assche, G. Permutation-Based Encryption, Authentication and Authenticated Encryption. In Proceedings of the Workshop on Symmetric Key Encryption, DIAC 2012, STMicroelectronics and NXP Semiconductors, Stockholm, Sweden, 5–6 July 2012.
21. Patgiri, R.; Nayak, S.; Muppalaneni, N.B. Is Bloom Filter a Bad Choice for Security and Privacy? In Proceedings of the 2021 International Conference on Information Networking (ICOIN), Jeju Island, Republic of Korea, 13–16 January 2021; pp. 648–653. [CrossRef]
22. Aguilera, M.K.; Ji, M.; Lillibridge, M.; McCormick, J.; Oertli, E.; Andersen, D.; Burrows, M.; Mann, T.; Thekkath, C.A. {Block-Level} Security for {Network-Attached} Disks. In Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST 03), San Francisco, CA, USA, 31 March–2 April 2003.
23. Tarkoma, S.; Rothenberg, C.E.; Lagerspetz, E. Theory and Practice of Bloom Filters for Distributed Systems. *IEEE Commun. Surv. Tutor.* **2012**, *14*, 131–155. [CrossRef]
24. Luo, L.; Guo, D.; Ma, R.T.; Rottenstreich, O.; Luo, X. Optimizing bloom filter: Challenges, solutions, and comparisons. *IEEE Commun. Surv. Tutor.* **2018**, *21*, 1912–1949. [CrossRef]
25. Srivastava, V.; Gupta, N.; Jati, A.; Bakshi, A.; Breier, J.; Chattopadhyay, A.; Debnath, S.K.; Hou, X. *Ascon-Sign: Submission to the NIST Post-Quantum Project*; Technical Report; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2023.
26. Magyari, A.; Chen, Y. Securing the Internet of Things with Ascon-Sign. *Internet Things* **2024**, *28*, 101394. [CrossRef]
27. LoRa Alliance. LoRaWAN Specification v1.0.3. 2018. Available online: https://lora-alliance.org/resource_hub/lorawan-specification-v1-0-3/ (accessed on 18 April 2024).
28. IBM. MQTT-SN Protocol Specification Version 1.2. 2013. Available online: https://groups.oasis-open.org/higherlogic/ws/public/document?document_id=66091 (accessed on 18 April 2024).
29. OASIS. MQTT Version 5.0. 2019. Available online: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed on 18 April 2024).
30. Koppuravuri, A.; Pasupuleti, H.; Gvk, S.; Bapat, J. A High Throughput ASCON Architecture for Secure Edge IoT Devices. In Proceedings of the 2024 37th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID), Kolkata, India, 6–10 January 2024; pp. 486–491. [CrossRef]
31. Khan, S.; Lee, W.K.; Hwang, S.O. Evaluating the Performance of Ascon Lightweight Authenticated Encryption for AI-Enabled IoT Devices. In Proceedings of the 2022 TRON Symposium (TRONSHOW), Tokyo, Japan, 7–9 December 2022; pp. 1–6.
32. Kandi, A.; Bakshi, A.; Gan, P.; Guilley, S.; Gerlich, T.; Breier, J.; Chattopadhyay, A.; Shrivastwa, R.R.; Martinásek, Z.; Bhasin, S. Side-Channel and Fault Resistant ASCON Implementation: A Detailed Hardware Evaluation (Extended Version). *Cryptol. Eprint Arch.* **2024**, 984. Available online: <https://eprint.iacr.org/2024/984> (accessed on 18 April 2024).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.